Design of a SIMD Execution Stage for 8-bit and Packed 4-bit Operations

Prathmesh Gabhane
BITS ID: 2025H1400103P
MEL G624 Advanced VLSI Architectures, Assignment 1

1. Problem

The aim is to design the execution stage of a small SIMD datapath. It takes two 8-bit inputs A and B, a 2-bit control fn that picks the operation, and a 1-bit control size that picks the data width. It produces two 8-bit outputs C and D.

When size is 0 the stage works on one 8-bit number in each input. When size is 1 each input holds two 4-bit numbers packed together (bits [7:4] and bits [3:0]), and the stage works on both halves at the same time. This is the same trick that CPU SIMD units and GPUs use to run many small operations in one wide ALU.

The three operations and the two widths give six cases in all. The required outputs are:

size	fn	D (output)	C (output)
0	add	A + B, carry out dropped	0
0	mul	low 8 bits of A x B	high 8 bits
0	eq	D[0] = (A == B), rest 0	0
1	add	D[3:0] = A[3:0]+B[3:0], D[7:4] = A[7:4]+B[7:4]	0
1	mul	A[3:0] x B[3:0]	A[7:4] x B[7:4]
1	eq	D[0] = low nibbles equal, D[4] = high nibbles equal	0

fn = 00 is add, 01 is multiply, 10 is equality. The code fn = 11 is not used; I made it give C = D = 0. All numbers are unsigned.

2. Main design idea

The easy way to build this is to make a separate circuit for every case and then pick the right one with a multiplexer. That works but it wastes a lot of area, because most of the circuits sit idle in any

given cycle. The better idea is that each 8-bit unit already contains the two 4-bit units inside it, so I only need a small change to make one unit do both jobs.

2.1 Addition: break the carry chain

An 8-bit adder is the same as two 4-bit adders, except that the carry out of bit 3 is passed into bit 4. If I stop that one carry from crossing, the adder splits into two independent 4-bit adders. So I gate the carry with the size bit:

```
carry_into_high_nibble = carry_out_of_low_nibble AND (NOT size)
```

With size = 0 the carry passes and the whole thing is an 8-bit adder. With size = 1 the high nibble gets a 0 carry-in and the two halves are separate. The cost of SIMD support in the adder is one AND gate.

2.2 Multiplication: reuse one multiplier

This is the important part. Split each operand into a high and low nibble:

```
A = 16*AH + AL        B = 16*BH + BL
```

Then the full product is:

```
A x B = 256*(AH*BH) + 16*(AH*BL + AL*BH) + (AL*BL)
```

The two middle terms, AH·BL and AL·BH, are the “cross” terms. For the packed 4-bit mode I only want AL·BL in D and AH·BH in C, and those are exactly the two terms that are left if I force the cross terms to zero. What remains is:

```
256*(AH*BH) + (AL*BL)
```

Now the useful fact: AL·BL is at most 15 × 15 = 225, which is less than 256. So the low product never reaches bit 8, and the two results do not overlap. The 16-bit product simply becomes the two nibble results placed side by side:

```
product = { AH*BH , AL*BL } = { C , D }
```

So one 8x8 multiplier does both modes. I do not even need a multiplexer on the output: in both modes C is the top 8 bits of the product and D is the bottom 8 bits. The only change is that the partial-product AND gates that belong to the cross terms get an extra input, (NOT size), so they are switched off in 4-bit mode:

```
if the AND gate belongs to a cross term:  pp = A[i] & B[j] & (NOT size)
otherwise:                                pp = A[i] & B[j]
```

Because this gating happens right at the inputs of the multiplier array, the rest of the multiplier does not know or care which mode it is in. That means I can use any multiplier style behind it without changing the idea.

2.3 Equality

For each nibble I XNOR the bits and AND them together, which gives one bit that is 1 when the nibble matches. The 8-bit compare is just the AND of the two nibble results. So this needs one extra AND gate over the plain 4-bit version.

3. Designs I compared

To find the best option I built four versions of the stage. All four have the same ports and give the same outputs, so one testbench and one synthesis script cover all of them.

- V1, naive: a separate unit for every case, selected by a mux. This is the baseline to compare against.
- V2, shared behavioural: the sharing ideas above, but written with the +, * and == operators so the tool chooses the adder and multiplier style.
- V3, structural (area): the sharing ideas built by hand from gates, using a ripple-carry adder and a simple shift-and-add array multiplier.
- V4, structural (fast): the same, but with a carry-select adder and a Dadda-tree multiplier with a Kogge-Stone final adder, which are the faster structures.

Comparing V1 to V2 shows what sharing saves. Comparing V2, V3 and V4 shows what hand-built fast structures gain.

4. Checking that it works

I wrote a golden model straight from the specification table above, and compared all four designs against it for every possible input. That is 256 values of A times 256 values of B, times 3 operations, times 2 size modes, which is 393216 input combinations for each design. The testbench reports zero mismatches for all four designs, so they are all functionally correct.

I also made a short testbench that prints a few hand-picked cases so the behaviour is easy to read. A sample of its output:

A	B	fn	size	C	D	meaning
FF	FF	mul	0	FE	01	255 x 255 = 65025 = 0xFE01
FF	FF	mul	1	E1	E1	15x15 = 225 = 0xE1 in both halves
3F	11	add	0	00	50	0x3F + 0x11 = 0x50
3F	11	add	1	00	40	low nibble F+1 does not carry into high nibble
77	77	eq	1	00	11	both nibbles equal, so D[0] and

A	B	fn	size	C	D	meaning
						D[4] set

The add case with the same inputs but different size shows the carry being broken, and the multiply case shows the cross terms being switched off.

5. Synthesis and characterisation

I synthesised all four designs with Yosys and mapped them to the Nangate 45 nm standard-cell library using ABC. For each design I recorded the number of cells (a measure of gate count), the cell area in square microns, and the critical-path delay in picoseconds (the speed). I ran the mapping twice: once asking the tool to save area, and once asking it to save delay. Reporting both is fairer, because a design that is small under the area setting may not be the fastest under the delay setting.

Area setting:

Design	Cells	Area (um^2)	Delay (ps)
V1 naive	589	583.1	1056
V2 shared	490	469.8	1075
V3 structural (area)	422	423.7	1550
V4 structural (fast)	441	438.9	880

Delay setting:

Design	Cells	Area (um^2)	Delay (ps)
V1 naive	773	879.1	535
V2 shared	639	739.7	564
V3 structural (area)	726	785.5	946
V4 structural (fast)	660	780.2	494

To see where the area and delay actually come from, I also synthesised the building blocks on their own (area setting):

Block	Area (um^2)	Delay (ps)
Adder, ripple carry	38.6	361
Adder, carry select	45.5	284
Equality comparator	17.3	102
Multiplier, shift-add array	360.7	1460
Multiplier, Dadda tree	385.2	792

6. Which design is best

A few clear points come out of the numbers.

The multiplier is the whole story. It is about 85 percent of the area and more than 90 percent of the delay of the stage. The adder and the comparator are tiny next to it. So effort spent making the adder faster barely changes the result, while the choice of multiplier decides almost everything.

Sharing is worth doing. Going from V1 to V2 cuts the area by about 19 percent (from 583 to 470 um^2) and the delay barely changes. The saving comes from using one 8x8 multiplier with gated cross terms instead of one 8x8 plus two 4x4 multipliers. There is no reason to keep the naive version.

Hand-built slow structures are not a win. V3 is the smallest design when the tool is told to save area, but it is also by far the slowest, because the shift-and-add array multiplier has a long carry path. Writing it by hand gained nothing over letting the tool do it.

The Dadda multiplier is the key improvement. Swapping the array multiplier for the Dadda tree cuts the multiplier delay almost in half (1460 to 792 ps) for only about 7 percent more area. This is what makes V4 both faster than every other version and smaller than the naive one.

So the recommendation depends on the need:

- If the only thing that matters is smallest area, pick V3 (423.7 um^2), but accept that it is slow.
- For almost any real use, pick V4. It is the fastest of all four and only about 4 percent larger than the smallest, so it gives the best balance of speed and area.
- If design time is short, V2 is a good middle option: it is simple to write and still 19 percent smaller than the naive design.

7. Conclusion

The packed-SIMD requirement can be met with very little extra hardware if it is designed in from the start. One AND gate breaks the adder into two, and switching off the cross-term AND gates turns a single 8x8 multiplier into two 4x4 multipliers whose results come out already placed in C and D. No output multiplexer is needed for the multiply case. All four designs were checked against a golden model over every input and are correct. Among them, the structural design with a carry-select adder and a Dadda-tree multiplier (V4) gives the best mix of speed and area and is my recommended design.

The Verilog source for the main design and the building blocks follows in the appendix.

Appendix: Verilog code

The first module is the shared design written with the +, * and == operators (V2). The next modules are the gate-level building blocks, followed by the structural design that connects them (V3). The faster design (V4) replaces the ripple-carry adder with a carry-select adder and the array multiplier with a Dadda-tree multiplier; both use the same carry-breaking and cross-term gating shown here, and are standard structures.

Shared design, written with operators (V2)

```
module simd_exec_v2_shared (
    input  wire [7:0] A,
    input  wire [7:0] B,
    input  wire [1:0] fn,
    input  wire       size,
    output wire [7:0] C,
    output wire [7:0] D
);
    localparam [1:0] FN_ADD = 2'b00,
                     FN_MUL = 2'b01,
                     FN_EQ  = 2'b10;

    wire nsize = ~size;

    // ---- shared adder: one carry chain, breakable at the nibble boundary --
    wire [4:0] sum_lo  = {1'b0, A[3:0]} + {1'b0, B[3:0]};
    wire       c_mid   = sum_lo[4] & nsize;
    wire [3:0] sum_hi  = A[7:4] + B[7:4] + c_mid;
    wire [7:0] add_res = {sum_hi, sum_lo[3:0]};

    // ---- shared multiplier: cross partial products gated off in SIMD mode -
    wire [7:0]  pLL  = A[3:0] * B[3:0];
    wire [7:0]  pHH  = A[7:4] * B[7:4];
    wire [7:0]  pLH  = (A[3:0] * B[7:4]) & {8{nsize}};
    wire [7:0]  pHL  = (A[7:4] * B[3:0]) & {8{nsize}};
    wire [8:0]  pmid = pLH + pHL;
    // {pHH,pLL} needs no adder: AL*BL <= 225 cannot carry into bit 8.
    wire [15:0] prod = {pHH, pLL} + {3'b0, pmid, 4'b0};

    // ---- shared comparator ------------------------------------------------
    wire eq_lo  = (A[3:0] == B[3:0]);
    wire eq_hi  = (A[7:4] == B[7:4]);
    wire eq_all = eq_lo & eq_hi;

    wire [7:0] eq_res = size ? {3'b0, eq_hi, 3'b0, eq_lo}
                             : {7'b0,  eq_all};

    // ---- output multiplexer -----------------------------------------------
    reg [15:0] cd;

    always @* begin
        case (fn)
            FN_ADD  : cd = {8'h00, add_res};
            FN_MUL  : cd = prod;              // serves BOTH modes unchanged
            FN_EQ   : cd = {8'h00, eq_res};
            default : cd = 16'h0000;          // fn = 2'b11 reserved
        endcase
    end

    assign C = cd[15:8];
    assign D = cd[7:0];
endmodule
```

Full adder and half adder

```
// Full adder : 3 -> 2 compressor
module fa (
    input  wire a,
    input  wire b,
    input  wire cin,
    output wire sum,
    output wire cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule

// Half adder : 2 -> 2 compressor
module ha (
    input  wire a,
    input  wire b,
    output wire sum,
    output wire cout
);
    assign sum  = a ^ b;
    assign cout = a & b;
endmodule
```

Ripple-carry adder

```
module adder_rca #(
    parameter W = 8
) (
    input  wire [W-1:0] a,
    input  wire [W-1:0] b,
    input  wire         cin,
    output wire [W-1:0] sum,
    output wire         cout
);
    wire [W:0] c;
    assign c[0] = cin;
```

```
    genvar i;
    generate
        for (i = 0; i < W; i = i + 1) begin : g_bit
            fa u_fa (
                .a    (a[i]),
                .b    (b[i]),
                .cin  (c[i]),
                .sum  (sum[i]),
                .cout (c[i+1])
            );
        end
    endgenerate

    assign cout = c[W];
endmodule
```

Breakable SIMD adder

```
module add_simd_rca (
    input  wire [7:0] a,
    input  wire [7:0] b,
    input  wire       size,
    output wire [7:0] sum
);
    wire c_lo;    // carry out of the low nibble
    wire c_mid;   // carry actually injected into the high nibble

    adder_rca #(.W(4)) u_lo (
        .a(a[3:0]), .b(b[3:0]), .cin(1'b0),
        .sum(sum[3:0]), .cout(c_lo)
    );

    // the one gate that breaks the carry chain
    assign c_mid = c_lo & ~size;

    adder_rca #(.W(4)) u_hi (
        .a(a[7:4]), .b(b[7:4]), .cin(c_mid),
        .sum(sum[7:4]), .cout(/* discarded per spec */)
    );
endmodule
```

Equality comparator

```
module eq_simd (
    input  wire [7:0] a,
    input  wire [7:0] b,
    output wire       eq_lo,   // A[3:0] == B[3:0]
    output wire       eq_hi,   // A[7:4] == B[7:4]
    output wire       eq_all   // A == B
);
    wire [7:0] xnor_ab = ~(a ^ b);

    assign eq_lo  = &xnor_ab[3:0];
    assign eq_hi  = &xnor_ab[7:4];
    assign eq_all = eq_lo & eq_hi;
endmodule
```

SIMD multiplier (gated partial products)

```
module mult_simd_array (
    input  wire [7:0]  a,
    input  wire [7:0]  b,
    input  wire        size,    // 0 = 8x8, 1 = two independent 4x4
    output wire [15:0] p
);
    wire nsize = ~size;

    // ---- gated partial products: ppf[j*8 + i] = a[i] & b[j] -----------------
    wire [63:0] ppf;

    genvar i, j;
    generate
        for (j = 0; j < 8; j = j + 1) begin : g_pp_row
            for (i = 0; i < 8; i = i + 1) begin : g_pp_bit
                if (((i < 4) && (j >= 4)) || ((i >= 4) && (j < 4))) begin : g_cross
                    // cross term - killed in 4-bit SIMD mode
                    assign ppf[j*8 + i] = a[i] & b[j] & nsize;
                end else begin : g_native
                    // belongs to AL*BL or AH*BH - always live
                    assign ppf[j*8 + i] = a[i] & b[j];
                end
            end
        end
    endgenerate

    // ---- accumulation -------------------------------------------------------
    // acc_j (9 bits) = (acc_{j-1} >> 1) + pp_row_j ,  p[j] = acc_j[0]
    // One 8-bit ripple-carry adder per row; seven adders in total.
    wire [8*9-1:0] accf;

    assign accf[0 +: 9] = {1'b0, ppf[0 +: 8]};   // row 0 needs no adder
```

```
    generate
        for (j = 1; j < 8; j = j + 1) begin : g_row
            wire [7:0] rsum;
            wire       rcout;
            adder_rca #(.W(8)) u_row (
                .a    (accf[(j-1)*9 + 1 +: 8]),   // acc_{j-1} >> 1
                .b    (ppf[j*8 +: 8]),
                .cin  (1'b0),
                .sum  (rsum),
                .cout (rcout)
            );
            assign accf[j*9 +: 9] = {rcout, rsum};
        end
    endgenerate

    generate
        for (j = 0; j < 8; j = j + 1) begin : g_p_low
            assign p[j] = accf[j*9];
        end
    endgenerate

    assign p[15:8] = accf[7*9 + 1 +: 8];
endmodule
```

Structural design that connects the blocks (V3)

```
module simd_exec_v3_struct_rca (
    input  wire [7:0] A,
    input  wire [7:0] B,
    input  wire [1:0] fn,
    input  wire       size,
    output wire [7:0] C,
    output wire [7:0] D
);
    localparam [1:0] FN_ADD = 2'b00,
                     FN_MUL = 2'b01,
                     FN_EQ  = 2'b10;

    wire [7:0]  add_res;
    wire [15:0] prod;
    wire        eq_lo, eq_hi, eq_all;

    add_simd_rca u_add (
        .a(A), .b(B), .size(size), .sum(add_res)
    );

    mult_simd_array u_mul (
        .a(A), .b(B), .size(size), .p(prod)
    );

    eq_simd u_eq (
        .a(A), .b(B), .eq_lo(eq_lo), .eq_hi(eq_hi), .eq_all(eq_all)
    );

    wire [7:0] eq_res = size ? {3'b0, eq_hi, 3'b0, eq_lo}
                             : {7'b0,  eq_all};

    reg [15:0] cd;

    always @* begin
        case (fn)
            FN_ADD  : cd = {8'h00, add_res};
            FN_MUL  : cd = prod;
            FN_EQ   : cd = {8'h00, eq_res};
            default : cd = 16'h0000;
        endcase
    end

    assign C = cd[15:8];
    assign D = cd[7:0];
endmodule
```